Teaching Statement: Learning Engineering Through Explanation, Experiment, and Design

Waseem Raza, PhD | Assistant Professor of Electrical/Computer Engineering | University of Scranton

What I want students to learn

An engineering student may know the equation for a communication channel or the syntax of a program and still be unsure what to do when the measured result disagrees with the prediction. My goal is to help students bridge that gap. I organize instruction around a progression: understand the principle, make a prediction, test it in code or a laboratory, and explain what the evidence means. The final explanation matters as much as obtaining a working result.

I taught Computer Programming, Communication Systems, and Wireless Communication to undergraduates in the University of Lahore's Department of Technology. At the University of Engineering and Technology, Taxila, I supported communications laboratories, assignments, grading, and instructional material. At COMSATS University Islamabad, I mentored students on engineering research and thesis projects. I have also tutored students at secondary and undergraduate levels outside formal faculty appointments. These experiences taught me to check prerequisite understanding early, break complex problems into workable steps, and give feedback that students can use on the next attempt.

Courses, laboratories, and curriculum development

My strongest teaching areas are programming, communication systems, wireless networks, signal analysis, and computational approaches to engineering problems. My ECE research also prepares me to develop instruction in digital signal processing, computer and communication networks, data-driven engineering, and applications of AI and ML. I would coordinate with colleagues to prepare for other essential EE and CE courses, using their learning objectives, laboratory equipment, prerequisites, and established course materials. Prior experience teaching communications does not automatically establish prior teaching in power electronics, electrical machines, circuit fabrication, or computer architecture; those require dedicated preparation.

At Scranton, I would be particularly interested in supporting learning across programming, signal processing, and intelligent systems. In a signal-processing laboratory, students could first predict the effects of sampling a waveform, then implement a filtering experiment using MATLAB or Python, and finally explain aliasing or noise effects using the acquired data. In communications, they could compare a theoretical path-loss model with a simulation and study why the results differ. Both examples can be made accessible to a range of preparation levels, with challenge questions for students ready to extend the analysis.

I am also interested in developing an upper-level elective or laboratory module in Applied AI for Sensors and Networked Systems. Students could build a small data pipeline, establish a simple baseline, measure false positives under changed conditions, and discuss the cost of putting a model on a resource-constrained device. The course would teach engineering judgment and experimentation rather than assume that using the newest neural network is itself a learning outcome. Scranton's computer engineering curriculum, which culminates in a robot-design capstone, offers natural opportunities to connect programming, sensing, and validation to hardware-centered design with colleagues who teach embedded and robotic systems.

Assessing and improving student learning

I would specify measurable outcomes at the start of each course or laboratory: deriving a relationship, writing and testing a program, interpreting a measurement, troubleshooting a system, or communicating an engineering tradeoff. I would use short concept checks and staged assignments before larger exams or projects. For laboratory reports, my rubric would assess theoretical reasoning, procedure, evidence, error analysis, and written interpretation. A failed experiment with a well-documented diagnosis can demonstrate more learning than an unexplained successful result.

I understand that the department serves multiple ABET-accredited engineering programs and expects continuous assessment. I would work with program coordinators to map assignments to relevant student outcomes, retain samples of student work and rubric data, identify recurring weaknesses, and use that evidence to revise exercises or prerequisite support. Changes would be assessed in the next offering rather than treated as completed simply because a revision was made.

Students arrive with different academic, linguistic, and computing backgrounds. I would provide tested setup instructions and clear expectations, use structured group roles so all students practice technical and communication tasks, and offer accessible notes, regular office hours, and early feedback. High standards are more effective when students can see a fair path toward meeting them.

Mentoring and formation

For capstone and independent projects, I would begin with a question that can be answered within the available time and resources. Students would establish requirements, reproduce a baseline, develop a test plan, and demonstrate results through a technical report or presentation. Regular meetings and version-controlled code would help them develop independence without leaving them unsupported. My experience guiding communications and software research projects provides a foundation for this work.

Scranton's Jesuit educational mission resonates with my view that professional formation includes ethical judgment and responsibility to the people affected by a system. When students assess a wearable sensor, for example, they should consider privacy and the implications of missed detections. When they optimize a network, they should consider tradeoffs among users, not just average performance. My aim is for graduates to be capable engineers who can explain and defend their decisions, recognize limitations, and continue learning after a course has ended.